INFORME DE GUÍA PRÁCTICA

I. PORTADA

Tema:	Aplicaciones Distribuidas
Unidad de Organización Curricular:	PROFESIONAL
Nivel y Paralelo:	6 - A
Alumnos participantes:	Ramírez Hinostroza Manuel Eduardo
	Jimenez Chango Saimol Steph
Asignatura:	Aplicaciones Distribuidas
Docente:	Ing. Alex Maigua, Mg.

II. INFORME DE GUÍA PRÁCTICA

2.1 Objetivos

General:

Desarrollar una arquitectura distribuida para una aplicación de gestión de reservas que garantice la escalabilidad, disponibilidad y confiabilidad de la información mediante una arquitectura orientada a servicios (SOA) implementada con Windows Communication Foundation (WCF).

Específicos:

- Implementar servicios independientes para autenticación, gestión de usuarios, vuelos y reservas siguiendo los principios de una arquitectura SOA.
- Desarrollar una aplicación cliente en Flutter capaz de consumir servicios distribuidos para la búsqueda y administración de vuelos y reservas.
- Garantizar la seguridad e integridad de la información mediante cifrado de contraseñas, autenticación basada en JWT y validaciones de concurrencia y formularios.

2.2 Modalidad

Presencial

2.3 Tiempo de duración

Presenciales: 4 horas

No presenciales: 0 horas

2.4 Instrucciones

Desarrollar una aplicación web de reservas de tickets aéreos que permita a los usuarios planificar y reservar vuelos, utilizando una arquitectura distribuida basada en SOA
con WCF. El sistema deberá contemplar alta disponibilidad y balanceo de carga para mejorar la escalabilidad y confiabilidad.

Registro de Usuario

- Permitir el registro de usuarios en el sistema.
- Registrar información personal de los clientes.
- Proteger la contraseña del usuario mediante método de encriptación (recomendado SHA256).
- Validar integridad de los datos almacenados

Inicio de Sesión

- Ingresar credenciales (usuario y contraseña).
- Verificar autenticación contra la base de datos.
- Redirigir a la página principal tras autenticación exitosa.
- Gestionar control de sesión

Búsqueda y Reserva de Vuelos

- Ingresar detalles del vuelo (origen, destino, fecha).
- Mostrar resultados disponibles.
- Permitir selección de vuelo.
- Completar proceso de reserva

Gestión de Reservas

- Acceder al historial de reservas.
- Visualizar detalles de cada reserva.
- Cancelar o modificar reservas cuando sea necesario

Arquitectura SOA - WCF

- Diseñar servicios independientes para cada proceso del sistema.
- Implementar servicios WCF para registro, autenticación, reservas y gestión.

- Separar capa de presentación, lógica de negocio y acceso a datos.
- Documentar el diseño arquitectónico implementado

2.5 Listado de equipos, materiales y recursos

- Internet
- Bases de datos disponibles en la biblioteca virtual de la Universidad.
- Bibliografía de la asignatura.
- Material disponible en el aula virtual de la asignatura.

TAC (Tecnologías para el Aprendizaje y Conocimiento) empleados en la guía práctica:

☐Plataformas educativas

☐Simuladores y laboratorios virtuales

☐Aplicaciones educativas

☐Recursos audiovisuales

☐Gamificación

☒Inteligencia Artificial

2.6 Actividades por desarrollar

Introducción

En la presente guía práctica se desarrolló una aplicación distribuida para la reserva de tickets aéreos, basada en una arquitectura orientada a servicios (SOA). El sistema fue diseñado bajo una estructura cliente-servidor desacoplada, compuesta por un frontend desarrollado en Flutter y un backend implementado con Flask, simulando la separación de servicios característica de arquitecturas distribuidas y SOA.

La aplicación permite el registro y autenticación de usuarios, búsqueda de vuelos, selección de asientos, administración de reservas y gestión de vuelos para usuarios administradores. Además, se implementaron mecanismos de seguridad mediante autenticación JWT, validación de formularios y control de concurrencia para evitar inconsistencias durante el proceso de reserva.

El backend se conectó a una base de datos PostgreSQL/Supabase, permitiendo la persistencia de la información y la correcta administración de entidades relacionadas con usuarios, vuelos,

reservas y asientos. Asimismo, se aplicaron principios de alta disponibilidad y escalabilidad mediante la separación de responsabilidades entre servicios y capas del sistema.

Desarrollo de la práctica

Diseño Arquitectónico

El sistema fue estructurado bajo una arquitectura distribuida basada en SOA, separando claramente:

- Capa de presentación (Flutter)
- Capa de lógica de negocio (Flask/API REST)
- Capa de acceso a datos (PostgreSQL/Supabase)

Cada módulo del sistema fue implementado como un servicio independiente encargado de gestionar funcionalidades específicas.

Desarrollo del Frontend

Se desarrolló una aplicación Flutter con interfaz Material Design personalizada, permitiendo:

- Registro e inicio de sesión.
- Consulta y filtrado de vuelos.
- Selección visual de asientos.
- Gestión de reservas.
- Administración de vuelos para usuarios con rol administrador.

Además, se implementó persistencia de sesión utilizando almacenamiento local y tokens JWT.

Desarrollo del Backend

El backend fue desarrollado con Flask, exponiendo endpoints REST para:

- Registro y autenticación de usuarios.
- Consulta de vuelos.
- Creación, edición y cancelación de reservas.
- Administración de vuelos y asientos.
- Validación de permisos de administrador.

También se configuró CORS para permitir la comunicación segura entre Flutter y la API.

Seguridad y Validaciones

Se aplicaron mecanismos de seguridad y validación mediante:

- Cifrado de contraseñas utilizando SHA256.
- Autenticación basada en JWT.
- Control de sesiones activas.

- Validación de formularios y restricciones de entrada.
- Control de concurrencia para evitar reservas simultáneas sobre un mismo asiento.
- Implementación de roles.

2.7 Resultados Obtenidos

- **Arquitectura Distribuida Funcional:** Se implementó una arquitectura cliente-servidor desacoplada basada en principios SOA, separando correctamente frontend, backend y base de datos.
- **Sistema de Autenticación Seguro:** Se desarrolló un sistema de registro e inicio de sesión utilizando autenticación JWT y cifrado de contraseñas mediante SHA256, garantizando la seguridad de las credenciales.
- **Gestión Integral de Reservas:** La aplicación permite crear, consultar, editar y cancelar reservas de vuelos de manera eficiente y segura.
- **Búsqueda Dinámica de Vuelos:** Se implementó un sistema de búsqueda con filtros por origen, destino y rango de fechas, facilitando la localización de vuelos disponibles.
- **Selección y Control de Asientos:** Se desarrolló un módulo visual para la selección de asientos, incluyendo control de concurrencia para evitar conflictos en reservas simultáneas.
- **Administración de Vuelos:** Se implementó un módulo exclusivo para administradores que permite registrar nuevos vuelos y gestionar la disponibilidad de asientos.
- **Persistencia y Validación de Datos:** La información del sistema se almacena en PostgreSQL/Supabase, aplicando validaciones de integridad y restricciones de entrada en formularios y servicios.
- **Comunicación Exitosa entre Servicios:** El frontend Flutter logró consumir correctamente todos los servicios expuestos por la API REST, garantizando sincronización y actualización de datos en tiempo real.
- **Implementación de Roles:** Solo el administrador del sistema puede crear nuevos vuelos.

En la sección **Anexos** se muestran las capturas de las pantallas principales de la aplicación.

2.8 Habilidades blandas empleadas en la práctica

☐ Liderazgo

☒ Trabajo en equipo

☐ Comunicación asertiva

☐ La empatía

☒ Pensamiento crítico

☐ Flexibilidad

☐ La resolución de conflictos

☐ Adaptabilidad

☒ Responsabilidad

2.9 Conclusiones

- La implementación de una arquitectura distribuida basada en SOA permitió separar adecuadamente las responsabilidades del sistema, facilitando la escalabilidad, mantenibilidad y reutilización de servicios.
- El uso de Flutter y Flask permitió desarrollar una solución multiplataforma eficiente, capaz de gestionar reservas aéreas mediante comunicación cliente-servidor segura y estructurada.
- La incorporación de mecanismos de autenticación JWT, cifrado de contraseñas y control de concurrencia fortaleció la seguridad e integridad de la información procesada por el sistema.

2.10 Recomendaciones

- Implementar balanceadores de carga y contenedores Docker para mejorar la disponibilidad y escalabilidad del sistema en ambientes de producción.
- Incorporar mecanismos de monitoreo y registro de eventos (logs) para facilitar la detección de errores y el análisis del rendimiento de los servicios distribuidos.
- Optimizar las consultas a la base de datos y aplicar mecanismos de caché para reducir tiempos de respuesta en búsquedas y consultas frecuentes.

2.11 Referencias bibliográficas

[1] Erl, T. (2016). Service-Oriented Architecture: Concepts, Technology, and Design. Pearson Education.

[2] Freeman, A. (2021). Flutter for Beginners. Packt Publishing.

[3] Grinberg, M. (2018). Flask Web Development. O'Reilly Media.

[4] Richardson, L., & Ruby, S. (2008). RESTful Web Services. O'Reilly Media.

[5] Stallings, W. (2017). Cryptography and Network Security. Pearson.

2.12 Anexos



Figura 1: Registro de Usuario



Figura 2: Inicio de Sesión



Figura 3: Búsqueda de Vuelos



Figura 4: Selección de Asientos



Figura 5: Reservas del Usuario



Figura 6: Filtro de Vuelos por Rango de Fechas



Figura 7: Asiento Reservado



Figura 8: Registro de Vuelo solo para Administrador